PROGRAMACIÓN WEB I · 4º GRADO EN INGENIERÍA INFORMÁTICA · CURSO 2026-27

Práctica 1 — Pedir y auditar

Auditoría de código generado por un asistente.

Sesión: 5 · **Modalidad:** individual · **Régimen de IA:** D · **Entrega:** campus virtual, antes de la sesión 6

Qué se pide

> **Le vas a pedir a un asistente de código que amplíe un servidor, y después vas a auditar lo que te dé.**
>
> Lo que se evalúa es la auditoría, no el código. Que el código generado tenga defectos es lo esperable: son el material de trabajo. Lo que se corrige es si los encontraste, cómo lo comprobaste y qué dices de ellos.

Una auditoría honesta que diga «esta línea no sé verificarla» vale más que un «todo correcto» que no lo esté. No hay ninguna ventaja en aparentar que encontraste algo que no encontraste, ni en arreglar el código antes de auditarlo: lo que se entrega es el código tal como salió.

Régimen D — qué significa

	Régimen B (sesión 4)	Régimen D (esta práctica)
La IA es…	un medio para hacer el trabajo	**el objeto que se evalúa**
Se evalúa…	el resultado	**tu análisis del resultado**
Trazabilidad	recomendada	**obligatoria**
Declaraciones	autoría + uso de IA	autoría + uso de IA

Sin la declaración de autoría y la declaración de uso de IA la entrega no se corrige.

El material de partida

Haz git clone del repositorio https://github.com/dperezalvarez/PW1-2026-sesion5.git

Fichero	Qué es
servidor.js	Un servidor HTTP con dos rutas escritas a mano. No sabe leer ficheros del disco.
public/index.html	Una página de verdad, con hoja de estilos e imagen enlazadas.
public/estilos.css	Los estilos de esa página. Si no se aplican, algo va mal con el tipo de contenido.
public/logo.svg	Una imagen. Tercer tipo de fichero distinto.
notas-privadas.txt	Está FUERA de public/. Nadie debería poder leerlo desde el navegador. Averigua si tu servidor lo permite.

Los seis pasos

Paso 1 · Ejecuta el código de partida

Antes de pedir nada. Arranca el servidor y comprueba que ves la portada en http://localhost:3000. Si esto no funciona, nada de lo que venga después funcionará.

```
node servidor.js
Escuchando en http://localhost:3000
```

Paso 2 · Pídeselo al asistente

Escribe exactamente esto, sin añadir nada:

```
Amplía este servidor para que sirva los ficheros
de la carpeta public desde el disco.
```

No le des pistas. Nada de «con seguridad», nada de «con validación», nada de «comprueba los errores». Lo que queremos ver es qué produce una petición normal.

> **Guarda estas tres cosas AHORA, no al final**
>
> 1. El prompt exacto que escribiste, incluidos los reintentos si los hubo.
> 2. El código que te devolvió, sin tocar. El original, no tu versión arreglada.
> 3. Qué asistente y qué modelo usaste.

Si te devuelve una solución con Express, no la aceptes: no lo tienes instalado y esta práctica es con el módulo http. Pídeselo otra vez diciéndolo, y anota también ese segundo intento — el ida y vuelta forma parte de la traza.

Paso 3 · Ejecútalo

Criterio de éxito: que http://localhost:3000 muestre la página con los estilos aplicados y el logo visible. Si se ve sin estilos o sin imagen, ya tienes tu primer hallazgo — no lo arregles todavía, anótalo.

Paso 4 · Audítalo con esta lista

Seis comprobaciones. Para cada una responde sí, no o «no sé», y explica cómo lo comprobaste. Las tres respuestas puntúan si están justificadas; lo que no puntúa es responder sin haber comprobado.

#	Comprobación	Pista
1	¿Arranca y sirve la página con estilos y logo?	Si el CSS no se aplica, el tipo de contenido está mal.
2	¿Qué pasa si pides un fichero que no existe?	Prueba /noexiste.html. ¿Un 404, o se cae el servidor?
3	**¿Puede alguien salirse de la carpeta public?**	El comando exacto está más abajo.
4	¿El Content-Type sale de la extensión o está fijo?	Busca un mapa de extensiones, o un literal repetido.
5	¿Existe cada método que usa?	Compruébalo en nodejs.org/api, no de memoria.
6	¿Sabrías explicar cada línea si te pregunto?	La que no sepas explicar es la que hay que mirar.

Cómo se comprueba la número 3

Con el servidor arrancado, abre otra terminal y ejecuta estos dos comandos. En Windows escribe curl.exe, no curl.

```
curl --path-as-is "http://localhost:3000/../notas-privadas.txt"

curl --path-as-is "http://localhost:3000/%2e%2e%2fnotas-privadas.txt"
```

> **Si en la respuesta aparece el contenido de notas-privadas.txt, tu servidor deja leer ficheros de todo el disco. Eso tiene nombre: path traversal.**
>
> Prueba las dos formas antes de responder que no. La primera funciona si el código concatena la ruta tal cual; la segunda hace falta si además decodifica la URL.

Por qué no basta con escribirlo en el navegador: el navegador elimina los .. de la ruta antes de enviar la petición, y curl hace lo mismo salvo que le pongas --path-as-is. Que no te funcione por la barra de direcciones no significa que tu servidor sea seguro: significa que tu cliente te está protegiendo. Un atacante no usa navegador.

Si tu código resultó no ser vulnerable, eso es un hallazgo, no un fracaso. Localiza la línea concreta que lo impide, explica por qué funciona, y compárala con el código de tu compañero en el paso siguiente.

Paso 5 · Compara con tu compañero

Pedisteis lo mismo con las mismas palabras. No os va a haber salido lo mismo. Anota en qué se diferencian los dos programas y qué conclusión sacas de que difieran.

Paso 6 · Escribe la entrega

Los apartados están justo debajo. Empieza en clase y termínala en casa.

Qué se entrega

Un documento (PDF o DOCX) con cinco apartados. No hace falta que sea largo; hace falta que sea concreto. Tres páginas bien hechas bastan.

1 · El prompt exacto

Copiado literal, incluidos los reintentos si los hubo. Indica qué asistente y qué modelo usaste.

2 · El código que te devolvió

Sin retocar. El original, no tu versión corregida. Si el asistente reescribió el fichero entero, incluye lo que quedó.

3 · La auditoría

Las seis comprobaciones, una por una. Para cada una: la respuesta, y qué hiciste para llegar a ella (el comando que ejecutaste, la URL que probaste, la página de la documentación que consultaste). Una respuesta sin método no cuenta.

4 · La comparación

En qué se diferenciaba tu código del de tu compañero, y qué te dice eso sobre cómo funcionan estas herramientas.

5 · Las declaraciones

Obligatorias. Sin ellas la entrega no se corrige.

- **Declaración de autoría.** Que el trabajo de análisis es tuyo.
- **Declaración de uso de IA.** Herramienta utilizada, finalidad y fase del trabajo en que se usó.

Cómo se evalúa

Qué se mira	Peso
La auditoría: cobertura de las seis comprobaciones y método empleado en cada una	**50 %**
El hallazgo del apartado 3 (path traversal) y su justificación, sea afirmativa o negativa	**20 %**
La comparación con el compañero y la conclusión que extraes	**15 %**
Trazabilidad completa: prompt, código original, herramienta y modelo	**15 %**

Lo que NO baja la nota

Que el código generado tenga defectos. Que no funcione a la primera. Que tu respuesta a una comprobación sea «no sé», si explicas qué intentaste. Nada de eso es culpa tuya y nada de eso se penaliza.

Lo que SÍ la baja

Entregar el código ya arreglado haciéndolo pasar por el generado. Responder a las comprobaciones sin haberlas hecho. Omitir el prompt o el código original. Entregar sin declaraciones.

La regla que sale de esta práctica

> *Nunca aceptes lo que no sabes verificar.*

Volveremos a este código en la sesión 18, cuando veamos el CWE Top 25: el fallo del apartado 3 tiene número propio, CWE-22, y no es una rareza — está entre los veinticinco más frecuentes del mundo real.